Consistency Models

Ibon De Mingo Arroyo
Universidad Autónoma de Madrid

Eloísa Denia García
Universidad Autónoma de Madrid

Matin Esmaeilzadeh
Universidad Autónoma de Madrid

Abstract

[PENDIENTE]

1 Introduction

Diffusion models [1, 2] are currently one of the most successful families of generative models for images. They gradually destroy the data by adding noise and train a neural network to reverse this process. The price of this quality is the cost of sampling: generating a single image requires starting from pure noise and removing it step by step, evaluating the network at every step. Ho et al. [1] use 1000 steps per sample. Formulating the process as an ordinary differential equation, the *probability flow ODE* [2], greatly reduces this number, but tens of network evaluations per image are still needed.

To address this problem, Song et al. [3] introduced *Consistency Models*. A consistency model learns to map any point on a trajectory of this ODE directly to its origin, that is, to the clean image at the end of the trajectory. As a result, it can generate a sample with a single network evaluation. When more compute is available, it can also chain several steps, adding noise back and removing it again, to improve quality. The paper proposes two ways to train them: *consistency distillation* (CD), which distills a pre-trained diffusion model that acts as a teacher, and *consistency training* (CT), which trains them from scratch, without a teacher.

In this project we reproduce this method at a small scale on MNIST [4] and measure what is gained and what is lost when the number of sampling steps is reduced. MNIST was chosen because it is light enough to train several complete models within the project deadline and without a GPU.

With respect to the initial proposal, the project has been extended in one direction: besides the diffusion teacher, we trained a second teacher based on Rectified Flow [5], and also distilled it into a consistency model. This allows us to study how the choice of teacher affects the resulting consistency model.

2 Objectives

The main objectives set in the project proposal are:

1. Train a diffusion model to serve as the teacher of the consistency model.
2. Distill the knowledge learned by the teacher into a faster consistency model.
3. Compare the consistency model with the diffusion model in terms of sampling speed and sample quality.
4. Analyze the results: the speed-up obtained, the quality of the samples generated with more or fewer steps and their diversity, as well as other technical issues of interest and the limitations found during the project.

Once these objectives were met, the proposal considered three extensions, in this order: training the model with consistency training and comparing it with the previous ones; adapting the architecture to color images with CIFAR-10 [6]; and developing a class-conditional consistency model able to generate a given digit.

All four main objectives have been met. Of the extensions, only the first one, consistency training, was carried out; color images and conditional generation were left out for lack of time. In addition, and following a suggestion made during the course, we trained a Rectified Flow teacher and distilled it as well, so that the diffusion teacher of the proposal can be compared with a flow-matching one.

3 Related work and state of the art

Diffusion models. Diffusion models were introduced by Sohl-Dickstein et al. [7] and became competitive with DDPM [1] and score-based models [8]. Song et al. [2] unified both views as stochastic differential equations and showed that every diffusion process has an associated deterministic probability flow ODE with the same marginal distributions. Karras et al. [9] later reorganized their design space (noise schedule, network preconditioning and ODE solver); their choices are the basis of consistency models and of our implementation.

Fast sampling. Several lines of work reduce the number of network evaluations. Training-free methods use better solvers for the probability flow ODE, such as DDIM [10], the second-order Heun sampler of EDM [9] or DPM-Solver [11], which reaches good quality in around 10 steps. Distillation methods train a student to imitate several teacher steps at once: progressive distillation [12] halves the number of steps in each round, down to 4 steps, at the cost of several rounds of training. Finally, flow matching [13] and Rectified Flow [5] learn a velocity field along nearly straight paths between noise and data, which can be integrated with fewer steps; Rectified Flow can be straightened further with *reflow*.

Consistency models. Consistency models [3] learn a direct map from any point of an ODE trajectory to its origin. They allow one-step generation and, unlike progressive distillation, are trained in a single stage, either by distillation (CD) or from scratch (CT). Song and Dhariwal [14] substantially improved CT with a pseudo-Huber loss and a new discretization curriculum. Later works apply the idea to latent text-to-image models [15], learn jumps between any two times of the trajectory [16], and move to continuous time to scale consistency models to 1.5 B parameters [17]. All of them rely on large GPU budgets; our goal is instead a small reproduction on CPU, in which we also compare a diffusion model and a Rectified Flow as teachers.

4 Methodology

We train two teachers on MNIST, a Variance Exploding (VE) diffusion model and a Rectified Flow (RF), and distill each of them into a consistency model. A third consistency model is trained from scratch with CT. All models use the same network, so that differences between them come from how they are trained and not from their capacity. Everything is implemented in PyTorch; the code is in the project repository.

4.1 Data and architecture

We use the 60 000 training images of MNIST, 28×28 in grayscale, with pixel values scaled to $[-1, 1]$ and no data augmentation. The 10 000 test images are only used in the evaluation (Section 4.6).

Every model is built on the same small U-Net [18]. The encoder has four convolutional levels with 32, 64, 128 and 256 channels, the last three with stride 2, and the decoder mirrors it with transposed convolutions and skip connections. Each level uses group normalization and the SiLU activation. The time t is encoded with 256 Gaussian random Fourier features followed by a linear layer, and this embedding is added to the feature maps after every convolution except the last one. The network has 1 115 425 parameters. The three uses of it differ only in how its output is read: the score network divides it by the noise level $\sigma(t)$, the velocity network takes it as it is, and the consistency models wrap it as described in Section 4.3.

4.2 Teacher models

In both teachers the data lie at $t = 0$ and the noise at $t = T = 1$, and the training times are drawn from $[\varepsilon, 1]$ with $\varepsilon = 10^{-5}$. Using the same time convention lets the consistency models be trained and sampled with the same code for either teacher.

VE diffusion. The forward process only adds Gaussian noise, $x_t = x_0 + \sigma(t)\, z$ with $z \sim \mathcal{N}(0, I)$ and

$$\sigma(t) = \sqrt{\frac{\sigma^{2t} - 1}{2 \ln \sigma}}, \qquad \sigma = 25, \tag{1}$$

which gives a noise standard deviation of about 9.85 at $t = 1$. The score network s_θ is trained with denoising score matching [2], minimizing $\mathbb{E}\,\|\sigma(t)\, s_\theta(x_t, t) + z\|^2$ with t uniform in $[\varepsilon, 1]$. We train it for 60 epochs with Adam, batch size 64 and a learning rate of 10^{-3} with cosine decay to zero, and keep an exponential moving average (EMA) of the weights with rate 0.999, which is the model we use. [PENDIENTE: confirmar que mnist_all_ve_ema.pth se entrenó con esta receta (valores por defecto de ve_training.py); no hay log de ese entrenamiento en logs/.] Samples are obtained by integrating the probability flow ODE

$$\frac{dx}{dt} = -\frac{1}{2}\, g(t)^2\, s_\theta(x, t), \qquad g(t) = \sigma^t, \tag{2}$$

from $t = 1$ to $t = \varepsilon$ with Heun's method on the time grid of Karras et al. [9] ($\rho = 7$), spaced in σ rather than in t. Each Heun step costs two network evaluations.

Rectified Flow. Rectified Flow [5] joins each image and a noise sample $x_1 \sim \mathcal{N}(0, I)$ with a straight line, $x_t = (1 - t)\, x_0 + t\, x_1$, and trains a velocity network v_θ to regress the constant velocity of that line:

$$\mathcal{L}_{\text{RF}} = \mathbb{E}\left\|v_\theta(x_t, t) - (x_1 - x_0)\right\|^2, \qquad t \sim \mathcal{U}[\varepsilon, 1]. \tag{3}$$

It is trained for 40 epochs with Adam, batch size 64 and a constant learning rate of 10^{-3}, again keeping an EMA of the weights with rate 0.999. Sampling integrates $dx/dt = v_\theta(x, t)$ from $t = 1$ to $t = 0$ with uniform Euler steps, one network evaluation each. We also tried Heun steps for this teacher, but they did not improve the samples at a low number of steps, so we kept Euler. We also tried the cosine learning-rate decay of the VE teacher: starting from the same learning rate, it gave practically the same samples as the EMA weights, so we kept a constant learning rate. The two teachers were therefore tuned separately and not trained with the same budget (60 epochs for VE, 40 for RF), which should be kept in mind when comparing them.

The learned paths are not exactly straight, because the lines of different image–noise pairs cross and the network can only learn their average. To straighten them we apply one round of *reflow* [5]: we draw 60 000 noise samples, map each one to an image by integrating the first model (1-RF, EMA weights) with 100 Euler steps, and retrain on these fixed pairs, whose straight lines cross much less than those of random pairs. The resulting 2-RF starts from the same weights and is trained for 10 epochs with a learning rate of $2 \cdot 10^{-4}$, keeping an EMA of the weights as before.

4.3 Consistency models

Parameterization. A consistency model $f_\theta(x_t, t)$ must map every point of an ODE trajectory to the same clean image, and it must satisfy the boundary condition $f_\theta(x, \varepsilon) = x$. As in the original paper, we enforce the boundary condition by construction:

$$f_\theta(x, t) = c_{\text{skip}}(t)\, x + c_{\text{out}}(t)\, F_\theta\big(c_{\text{in}}(t)\, x,\ t\big), \tag{4}$$

where F_θ is the U-Net of Section 4.1 and

$$c_{\text{skip}}(t) = \frac{\sigma_{\text{d}}^2}{(\sigma(t) - \sigma(\varepsilon))^2 + \sigma_{\text{d}}^2}, \quad c_{\text{out}}(t) = \frac{\sigma_{\text{d}}\,(\sigma(t) - \sigma(\varepsilon))}{\sqrt{\sigma(t)^2 + \sigma_{\text{d}}^2}}, \quad c_{\text{in}}(t) = \frac{1}{\sqrt{\sigma(t)^2 + \sigma_{\text{d}}^2}}. \tag{5}$$

At $t = \varepsilon$, $c_{\text{skip}} = 1$ and $c_{\text{out}} = 0$, so the output is exactly x. $\sigma(t)$ is the noise level of the teacher: the VE $\sigma(t)$ above, or $\sigma(t) = t$ for RF. We keep $\sigma_{\text{d}} = 0.5$, the value of the original paper, although the standard deviation of MNIST in $[-1, 1]$ is closer to 0.62. The input scaling c_{in}, taken from EDM [9], was missing in our first version; without it, the network receives inputs with a standard deviation of up to 10 with the VE teacher.

Consistency distillation. We follow Algorithm 2 of Song et al. [3]. The interval $[\varepsilon, T]$ is discretized into $N = 150$ points with the Karras spacing, in σ for VE and directly in t for RF. At each iteration and for each image x_0 of the batch, we pick two adjacent points $t_n < t_{n+1}$, noise the image to $x_{t_{n+1}}$ with the forward process of the teacher, and take one step of the teacher's ODE solver back to $\hat{x}_{t_n}$ (Heun for VE, Euler for RF). The two points then lie, approximately, on the same trajectory, and the loss asks the model to give the same output at both:

$$\mathcal{L}_{\text{CD}} = d\big(f_\theta(x_{t_{n+1}}, t_{n+1}),\ f_{\theta^-}(\hat{x}_{t_n}, t_n)\big). \tag{6}$$

The target network f_{θ^-} is an EMA of f_θ with rate $\mu = 0.95$ and does not receive gradients. For d we use the pseudo-Huber distance $d(a, b) = \sqrt{\|a - b\|^2 + c^2} - c$ with $c = 0.00054\sqrt{784} \approx 0.015$ [14], which is less sensitive to outliers than the squared ℓ_2 distance; the best results of the original paper use LPIPS instead. We train for 40 000 iterations with Adam, a learning rate of $4 \cdot 10^{-4}$ and batch size 64. Besides the target network, we keep a slower EMA of the weights (rate 0.9999), which is the one saved and used for sampling.

The value of N is the main departure from the paper, which uses $N = 18$. The network only sees the N grid times during training, and with $N = 18$ our small U-Net did not generalize to the times in between: one-step samples, always taken at $t = T$, looked fine, but multistep sampling, which re-noises to other times, produced unrecognizable images. With $N = 150$ this problem disappeared.

We distill three models: CM-VE from the VE teacher, CM-RF from the 1-RF and CM-RF (reflow) from the 2-RF, always from the EMA weights of the teacher. [PENDIENTE: confirmar si CM-VE se destiló de mnist_all_ve.pth o de mnist_all_ve_ema.pth.] To check the effect of the teacher's EMA, we also distilled the 1-RF from its raw weights.

Consistency training. CT (Algorithm 3 of the original paper) trains the model without a teacher, and we only implemented it for the VE process. The teacher step is replaced by using the same noise for both points of the pair: $x_{t_{n+1}} = x_0 + \sigma(t_{n+1})\, z$ and $x_{t_n} = x_0 + \sigma(t_n)\, z$. In expectation, both lie on the same trajectory of the probability flow ODE. The loss is the same as in CD, but N and μ now follow the curriculum of the paper: at iteration k of K,

$$N(k) = \left\lceil \sqrt{\frac{k}{K}\big((s_1 + 1)^2 - s_0^2\big) + s_0^2} - 1 \right\rceil + 1, \qquad \mu(k) = \exp\left(\frac{s_0 \ln \mu_0}{N(k)}\right), \tag{7}$$

with $s_0 = 2$, $s_1 = 150$ and $\mu_0 = 0.95$, so that training starts with a coarse grid and refines it. The optimizer, distance and batch size are the same as in CD. [PENDIENTE: número de iteraciones usado en CT (por defecto 20 000, o fijado con –budget-minutes).]

4.4 Sampling from consistency models

To generate an image in one step, we draw $x_T \sim \mathcal{N}(0, \sigma(T)^2 I)$, with $\sigma(T) \approx 9.85$ for VE and $\sigma(T) = 1$ for RF, and return $f_\theta(x_T, T)$. This costs a single network evaluation.

With more steps we use Algorithm 1 of the original paper. After the first prediction $x \leftarrow f_\theta(x_T, T)$, for each time $\tau_1 > \cdots > \tau_{n-1}$ we add noise back to x and denoise it again in one jump, $x \leftarrow f_\theta(x_\tau, \tau)$, so that n steps cost n evaluations. Every prediction is clamped to $[-1, 1]$ before adding noise. The noise has to be added with the forward process of the teacher, because x_τ must look like the inputs the model saw during training:

$$\text{VE: } x_\tau = x + \sqrt{\sigma(\tau)^2 - \sigma(\varepsilon)^2}\ z, \qquad \text{RF: } x_\tau = (1 - \tau)\, x + \tau\, z, \qquad z \sim \mathcal{N}(0, I). \tag{8}$$

The VE formula is the one of the paper. For RF the current prediction also has to be scaled down by $1 - \tau$, since in the RF forward process the image fades out as the noise grows.

The times τ_i have a large effect on the result. The paper chooses them with a greedy search that minimizes FID, which we did not do. For VE we take the interior points of a Karras grid in σ with $n + 1$ points, which gives $\tau = 0.16$ ($\sigma \approx 0.53$) for two steps and $\tau = 0.60, 0.16, 0.004$ ($\sigma \approx 2.65, 0.53, 0.07$) for four. The same rule is not useful for RF: for two steps it gives $\tau = 0.027$, which adds so little noise that the second step hardly changes the sample. For RF we therefore space the times uniformly from $\tau_{\max}$ down to ε, so that four steps use $\tau = 0.5, 0.33, 0.17$ with $\tau_{\max} = 0.5$. We chose this value by inspecting CM-RF samples with two and four steps for $\tau_{\max}$ between 0.1 and 0.9 (Figure 5 in the appendix): up to 0.3 the samples are almost the same as with one step, and from 0.7 on the model regenerates the digit from heavy noise and many digits change or break.

4.5 Differences from the original paper

Besides the much smaller scale (a 28×28 grayscale dataset, a network of 1.1 M parameters and 40 000 iterations with batch size 64), our implementation differs from that of Song et al. [3] in the following points:

- **Teachers.** The paper distills an EDM model [9]. Our diffusion teacher is the VE SDE of Song et al. [2], indexed by $t \in [\varepsilon, 1]$ instead of by σ, so the Karras grid is built in σ and mapped back to t. The Rectified Flow teacher is not part of the paper.
- **Discretization.** We use $N = 150$ points instead of 18 (Section 4.3).
- **Distance.** We use the pseudo-Huber distance of the improved version [14] instead of LPIPS or ℓ_2.
- **Parameterization for RF.** The coefficients c_{skip}, c_{out} and c_{in} come from EDM and assume $x_t = x_0 + \sigma(t)\, z$. In Rectified Flow the image is also scaled by $1 - t$, so with $\sigma(t) = t$ they are a reasonable choice but not the optimal one. We kept them so that both teachers share the same consistency model; deriving coefficients for the RF interpolation is left for future work.
- **Sampling.** Re-noising follows the RF forward process for the RF models, and the times τ_i are not optimized for FID (Section 4.4).
- **Consistency training.** We use the curriculum of the original paper with the pseudo-Huber distance, but not the other changes of the improved version, such as removing the EMA from the target network. CT is only implemented for the VE process.
- **Evaluation.** Instead of FID we use a Fréchet distance on the features of our own MNIST classifier (Section 4.6).

4.6 Evaluation protocol

Reference classifier. Metrics based on Inception, such as FID, are designed for natural images, so we trained a small CNN on MNIST instead: two 3×3 convolutional blocks (32 and 64 channels, ReLU and 2×2 max-pooling), a hidden dense layer of 128 units and a 10-way output, with 421 642 parameters in total. It was trained for 10 epochs with Adam (learning rate 10^{-3}, batch size 128) and reaches 99.00% accuracy on the MNIST test set.

Quality and diversity metrics. Our main quality metric is FD-MNIST: the Fréchet distance between Gaussians fitted to the 128-dimensional penultimate-layer features of generated and real images. It is computed exactly like FID [19], with our classifier in place of Inception and 10 000 MNIST training images as the real reference. We also report the fraction of *recognizable* samples, those classified with a top-1 probability of at least 0.9, but only as a secondary metric: the classifier is over-confident outside the data distribution and assigns a digit with such confidence to 61% of pure-noise images. Diversity is measured by the entropy of the distribution of predicted digits, whose maximum, $\ln 10 \approx 2.30$, means that all digits are equally frequent. Since the Fréchet distance depends on the number of samples, every configuration (model and number of steps) is evaluated on the same $N_{\text{eval}} = 2000$ samples, generated from the same initial noise within each family (VE or RF). As references, we also evaluate 2000 real test images, which give the best achievable values, and 2000 pure-noise images.

Timing. We measure the wall-clock time needed to generate $N \in \{1, 10, 100, 500\}$ samples, averaged over 3 repetitions after a warm-up run, and report the time per image at $N = 500$. The speed-up of each configuration is computed with respect to the teacher of its family with the most steps. We also report the number of network function evaluations (NFE) per sample, which, unlike time, does not depend on the hardware. All measurements were made on the same CPU.

5 Results and discussion

[PENDIENTE]

5.1 Quality versus number of steps

[PENDIENTE]

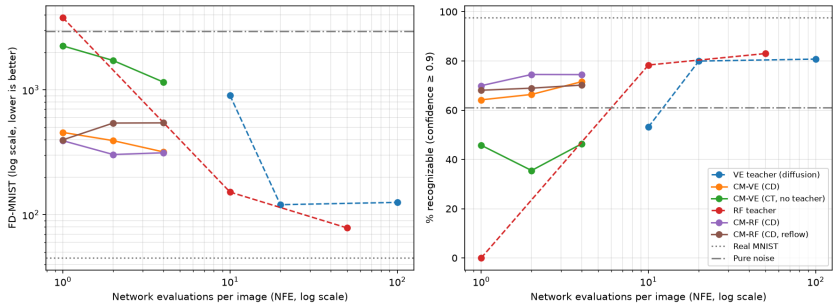


Figure 1: Sample quality versus number of network function evaluations (NFE): FD-MNIST (left, lower is better) and percentage of recognizable samples (right). Dashed lines are teachers and solid lines consistency models; gray lines mark the real-MNIST and pure-noise references.

5.2 Sampling speed

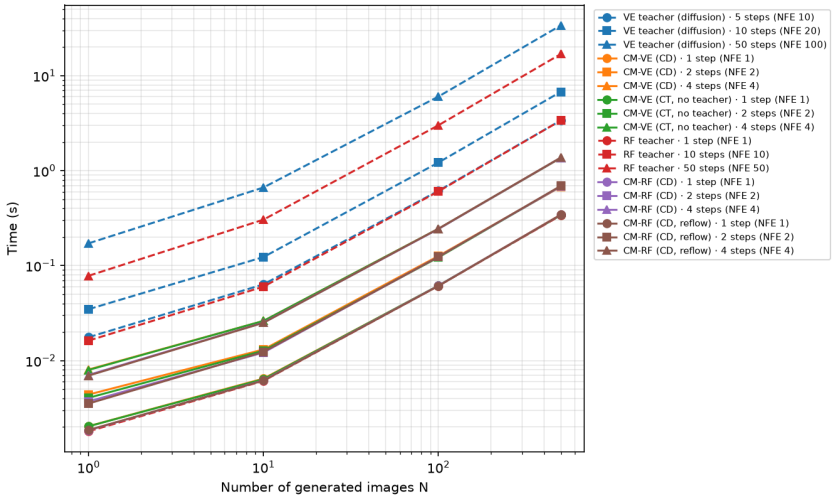


Figure 2: Sampling time versus number of generated samples N (log-log scale). Dashed lines are teachers and solid lines consistency models.

[PENDIENTE]

5.3 Diversity

[PENDIENTE]

5.4 Visual inspection

5.5 Discussion and limitations

[PENDIENTE]

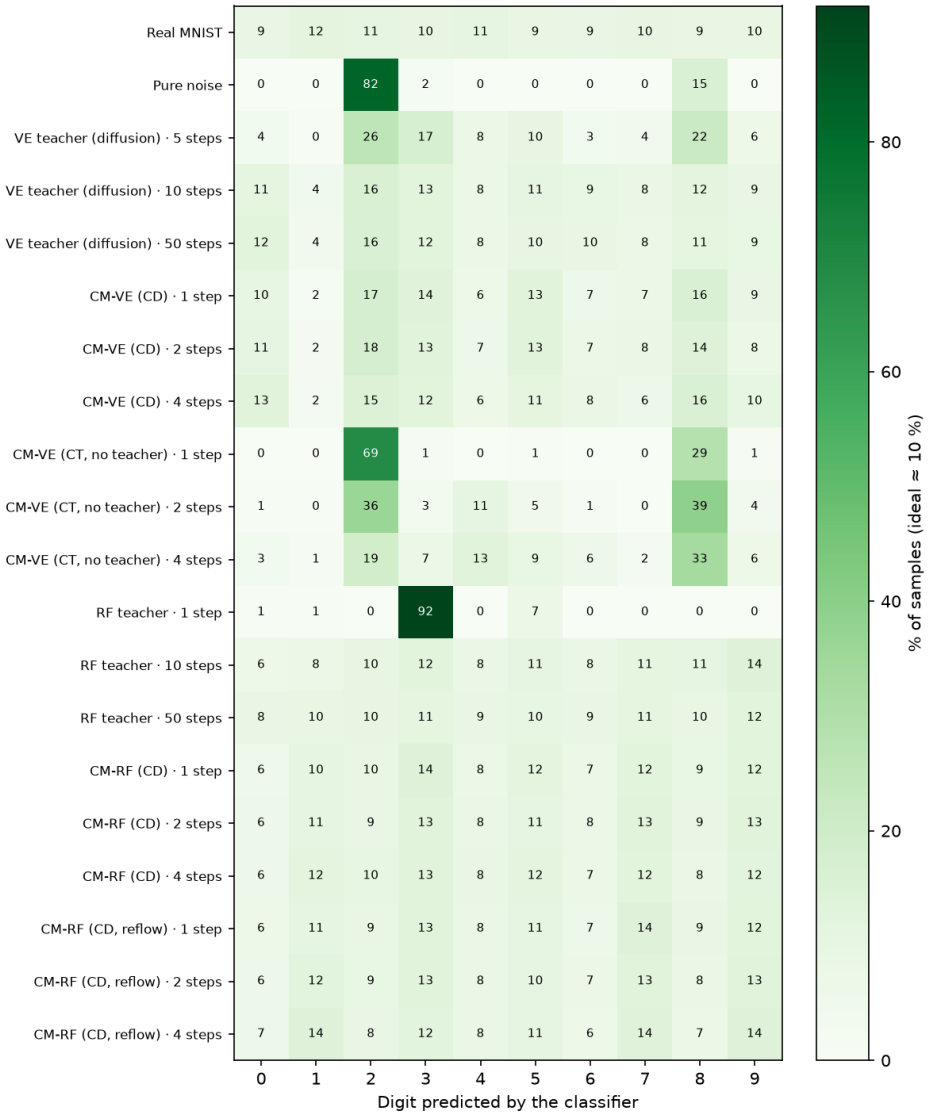


Figure 3: Distribution of the digits predicted by the reference classifier (percentage of samples per digit; a perfectly balanced generator gives 10% each).

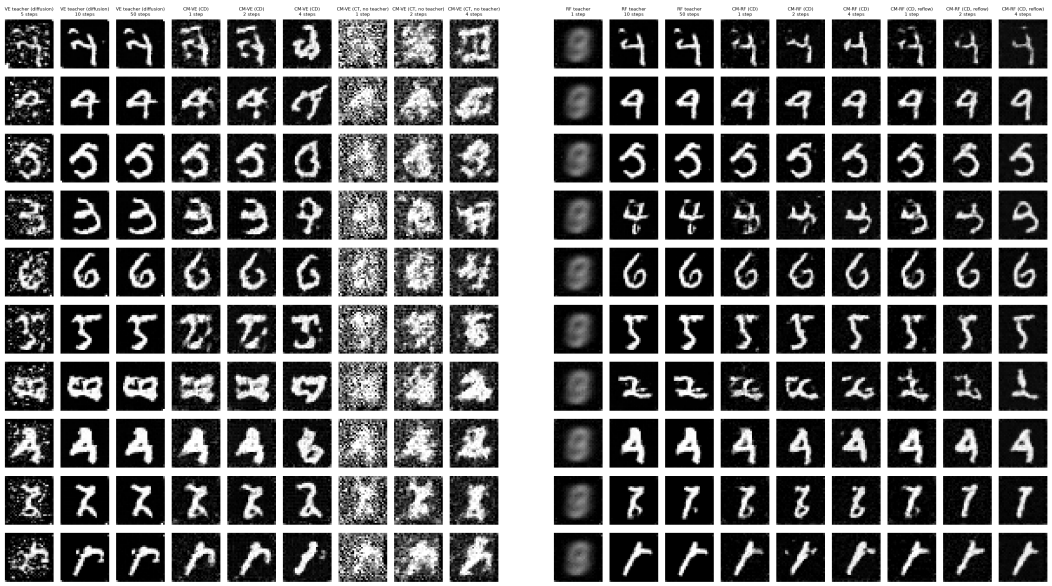


Figure 4: Samples from the VE family (left) and the RF family (right). Each column is a configuration and each row starts from the same initial noise.

6 Conclusions and future work

[PENDIENTE]

References

[1] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, volume 33, pages 6840–6851, 2020.

[2] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations*, 2021.

[3] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *PMLR*, pages 32211–32252, 2023.

[4] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[5] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations*, 2023.

[6] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

[7] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *PMLR*, pages 2256–2265, 2015.

[8] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[9] Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

[10] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations*, 2021.

[11] Cheng Lu, Yuhao Zhou, Fan Bao, Jianfei Chen, Chongxuan Li, and Jun Zhu. DPM-Solver: A fast ODE solver for diffusion probabilistic model sampling in around 10 steps. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

[12] Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models. In *International Conference on Learning Representations*, 2022.

[13] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023.

[14] Yang Song and Prafulla Dhariwal. Improved techniques for training consistency models. In *International Conference on Learning Representations*, 2024.

[15] Simian Luo, Yiqin Tan, Longbo Huang, Jian Li, and Hang Zhao. Latent consistency models: Synthesizing high-resolution images with few-step inference. *arXiv preprint arXiv:2310.04378*, 2023.

[16] Dongjun Kim, Chieh-Hsin Lai, Wei-Hsiang Liao, Naoki Murata, Yuhta Takida, Toshimitsu Uesaka, Yutong He, Yuki Mitsufuji, and Stefano Ermon. Consistency trajectory models: Learning probability flow ODE trajectory of diffusion. In *International Conference on Learning Representations*, 2024.

[17] Cheng Lu and Yang Song. Simplifying, stabilizing and scaling continuous-time consistency models. In *International Conference on Learning Representations*, 2025.

[18] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, volume 9351 of *LNCS*, pages 234–241, 2015.

[19] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. GANs trained by a two time-scale update rule converge to a local Nash equilibrium. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

A Technical appendices

Re-noising time for RF. Figure 5 shows the sweep used to choose $\tau_{\max}$ for multi-step sampling with the RF consistency models (Section 4.4). It was generated with `system/tests/cm_tau_sweep.py` on CM-RF.

[PENDIENTE: resto de apéndices.]

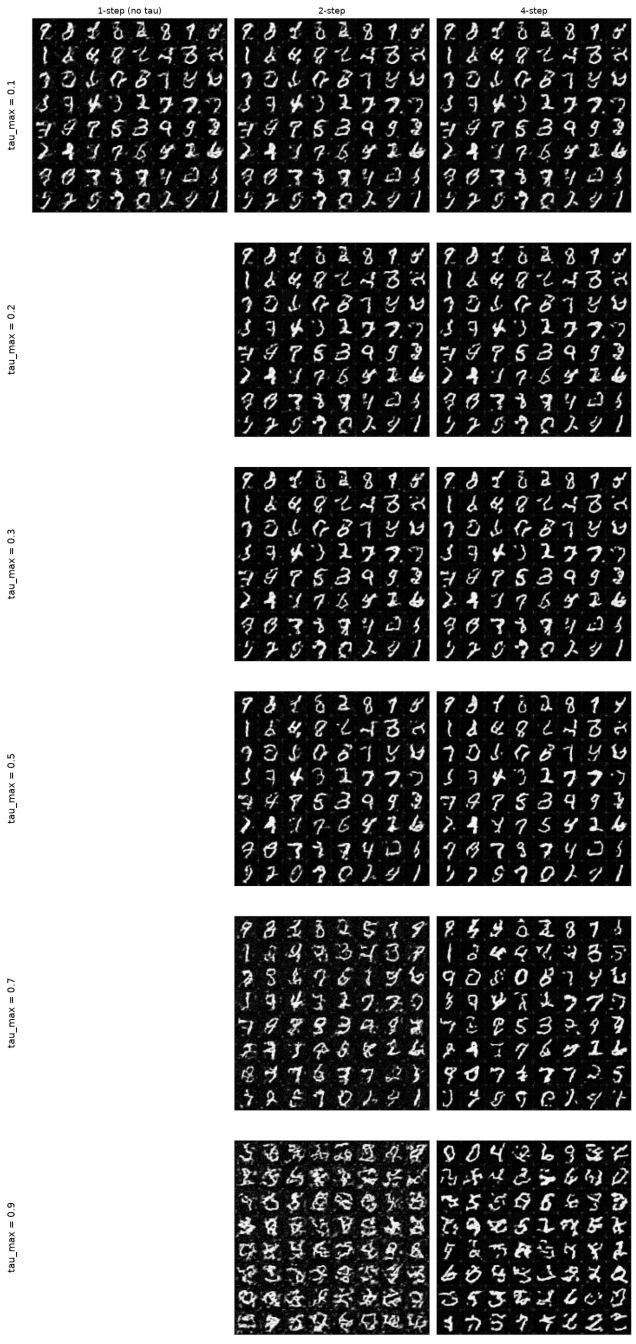


Figure 5: CM-RF samples for different values of $\tau_{\max}$. Each row is a value of $\tau_{\max}$ (0.1, 0.2, 0.3, 0.5, 0.7 and 0.9) and each column a number of steps (2 and 4); the panel at the top left is the one-step result, which does not depend on $\tau_{\max}$. Every panel shows the same 64 initial noises.

Author contributions

[PENDIENTE]

Declaration regarding the use of AI tools

[PENDIENTE]